VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590 018.

Group Activity

On

" INFORMATION & NETWORK SECURITY"

Submitted in partial fulfilment of the for the award of the degree of

Bachelor of Engineering

In

Information Science and Engineering

Submitted by

STUDENT NAME **USN**

STUDENT NAME **USN**

Guided by

Mr.Anoop N Prasad

Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

B.G.S COLLEGE OF ENGINEERING & TECHNOLOGY

BENGALURU – 560086

2026-2027

||Jai Sri Gurudev ||
BGSKH Education Trust (R.) – A unit of Sri Adichunchanagiri Shikshana Trust(R.)
BGS College of Engineering and Technology
Mahalakshmipuram, West of Chord Road, Bengaluru-560086
(Approved by AICTE, New Delhi and Affiliated to VTU, Belagavi)

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

Certificate

This is to certify that **STUDENT NAME (USN) , STUDENT NAME (USN), Information Science and Engineering** Department has successfully completed the Information & Network Security Activity-Based Learning Implementation and Source Code entitled **"DYNAMIC ONE-TIME PASSWORD AUTHENTICATION SCHEME WITH EXPIRY AND SINGLE-USE VERIFICATION"** in partial fulfillment for the requirement of the award of degree of Bachelor of Engineering **in Information Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the academic year 2026-27. The Information & Network Security mini project report has been approved as it satisfies the academic requirements in respect of the Information & Network Security implementation project work prescribed for the said degree.

Signature of the Guide

Mr.Anoop N Prasad

Signature of the HOD

Dr. Chaitra Naveen

ACKNOWLEDGEMENT

We sincerely thank **Visvesvaraya Technological University** for providing a opportunity to carry out a Information & Network Security implementation project entitled "**DYNAMIC ONE-TIME PASSWORD AUTHENTICATION SCHEME WITH EXPIRY AND SINGLE-USE VERIFICATION".**

We thank **Dr.Kiran P,** Principal, BGS College of Engineering & Technology, Bengaluru for providing all the necessary support to carry out the project.

We would like to place on record our regards to **Dr.Chaitra Naveen,** Head of the Department, Information Science and Engineering for his continued support.

We would also like to thank our guide, **Mr.Anoop N Prasad**, Assistant Professor, Dept. of ISE for his support and valuable guidance towards successful completion of the Information & Network Security implementation project.

We would be failing in our duty if we do not thank the faculty members, batch mates, lab staffs, technicians and family members for their constant support and guidance.

Student Name

USN

CONTENTS

1. Certificate

2. Acknowledgement

3. Abstract..... ii

4. List of Figures..... iii

5. List of Tables..... iv

6. Introduction..... 1

7. Problem Statement and Objectives..... 2

8. Software and Hardware Requirements..... 3

9. Algorithm / Methodology and Flowchart..... 4

10. Implementation and Source Code..... 8

11. Testing, Results and Screenshots..... 11

12. Security Analysis..... 15

Conclusion..... 17

References..... 18

Contribution of Group Members..... 19

Appendix: Complete Source Code..... 20

INS ACTIVITY-BASED LEARNING REPORT

Course Code: BIS703 **Course: Information & Network Security**

Project: Implement a dynamic one-time-password authentication scheme with expiry and single-use verification

ABSTRACT

Static passwords suffer from two weaknesses identified in Module 3 of BIS703: they are easy to steal (vulnerability) and, once stolen, they can be reused for as long as they remain valid (repeatability). This project implements a dynamic one-time-password (OTP) authentication scheme that removes both weaknesses. Following the challenge-response dynamic password scheme described in the course notes, each user holds a PIN-protected software token that stores a 256-bit secret key shared with the authentication server. For every login the server issues a fresh 128-bit random challenge together with a per-user sequence number and an expiry time. The token computes a six-digit OTP using HMAC-SHA256 followed by the RFC 4226 dynamic-truncation step, binding the challenge, counter and expiry together so none of them can be altered. The server stores only a salted hash of the expected OTP and accepts a submission only if it is correct, arrives before its expiry time, has never been used before and is within a limit of three attempts; repeated failures lock the account. A time-based (TOTP, RFC 6238) mode with clock-drift tolerance and per-step replay protection is also provided. The system is written in Python 3 using only the standard library (hmac, hashlib, secrets). It was evaluated with a nine-scenario demonstration and nineteen automated unit tests covering normal login, replay, expiry boundaries, malformed and tampered input, brute force, stolen tokens and clock drift. All tests passed, showing that every OTP is valid exactly once and only within its window of acceptability.

Keywords: one-time password, entity authentication, freshness, HMAC, challenge-response, TOTP, replay attack.

LIST OF FIGURES

Figure 1: Block diagram of the dynamic OTP authentication system.....4

Figure 2: OTP generation using HMAC-SHA256 and dynamic truncation.....5

Figure 3: Sequence diagram of one authentication attempt.....6

Figure 4: Flowchart of server-side OTP verification.....7

Figure 5: Registration, normal login and replay rejection.....12

Figure 6: Expired OTP, wrong and malformed OTPs, attempt limit and tampered expiry.....12

Figure 7: Stolen token without PIN, and TOTP mode single-use behaviour.....13

Figure 8: Brute-force attempts leading to account lockout, with audit log.....13

Figure 9: Automated unit test run: 19 tests, all passed.....14

LIST OF TABLES

Table 1: Software requirements.....3

Table 2: Hardware requirements.....3

Table 3: Project modules.....8

Table 4: Test cases and results.....11

Table 5: Threats and countermeasures.....15

Table 6: Contribution of group members.....19

CHAPTER 1

INTRODUCTION

1.1 Background

Entity authentication is the assurance that a given entity is involved and currently active in a communication session. As the Module 3 notes stress, it really involves two things: the **identity** of the entity making the claim and the **freshness** of the evidence, so that an attacker cannot capture the information used in one authentication session and replay it later to impersonate the genuine user.

The most common mechanism, the password, provides identity information but no freshness. A password is exactly the same every time it is used, so anyone who obtains it by shoulder-surfing, phishing, keylogging or a database breach can reuse it until it is changed. The notes summarise these problems as limited length, repeatability and vulnerability.

A **dynamic password scheme**, also called a **one-time password (OTP) scheme**, keeps the familiar idea of typing a short code but replaces the fixed password with a password function, a cryptographic algorithm keyed with a secret K, applied to an input that changes on every attempt. The value transmitted is therefore different every time, and an observed value is useless afterwards. The notes list three ways of making the input fresh, each of which is used in this project:

- **Clock-based**: the user and the server share synchronised time; the OTP is valid only within a window of acceptability.
- **Sequence numbers**: both sides maintain a counter that increases with every authentication.
- **Nonce-based (challenge-response)**: the server sends a random challenge and the user's token computes a cryptographic response to it.

1.2 Relevance and practical applications

OTPs are the basis of most two-factor authentication deployed today. Typical uses are:

- Internet banking and UPI transaction confirmation (OTP sent to a registered mobile number).
- Authenticator apps such as Google Authenticator or Microsoft Authenticator (TOTP, RFC 6238).
- Hardware tokens such as RSA SecurID and bank card readers (EMV CAP) that implement challenge-response.
- VPN and remote access, e-mail account recovery, and government portals such as Aadhaar e-KYC.

In all of these, security depends on three properties this project implements: an OTP must be unpredictable, short-lived and usable only once.

CHAPTER 2

PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem statement

Implement a dynamic one-time-password authentication scheme with expiry and single-use verification.

The system must let a registered user prove their identity to a server using a password that changes on every login, that becomes invalid after a short time, and that the server will accept at most once, so that a captured or guessed OTP cannot be used to gain access.

2.2 Objectives

1. Generate six-digit OTPs from a 256-bit shared secret using HMAC-SHA256 and dynamic truncation, with a fresh 128-bit random challenge for every login (target: no repeated OTP observed across 50 consecutive logins).
2. Enforce **expiry**: reject any OTP submitted after its validity period (60 seconds by default), verified at the exact boundary second.
3. Enforce **single use**: a correctly verified OTP must be rejected on every subsequent submission (replay rejected in 100 % of test cases).
4. Resist guessing: limit each challenge to 3 attempts and lock the account for 300 seconds after 5 failed attempts, compare values in constant time and never store the plain OTP on the server.
5. Provide two-factor protection with a PIN-protected token, and a time-based (TOTP) mode that tolerates ±30 seconds of clock drift while still refusing reuse.

2.3 Expected output

A Python library with a token (claimant) and a server (verifier), a command-line demonstration that shows ACCEPTED or REJECTED with a reason for each scenario, and an automated test suite whose results are reported in Chapter 6.

CHAPTER 3

SOFTWARE AND HARDWARE REQUIREMENTS

3.1 Software requirements

Table 1: Software requirements

Component	Used in this project	Purpose
Programming language	Python 3.9 or later (tested on 3.12)	Readable, cross-platform, and has strong cryptographic primitives in its standard library.
hmac (standard library)	hmac.new, hmac.compare_digest	Computes HMAC-SHA256 (the password function) and performs constant-time comparison to prevent timing attacks.
hashlib (standard library)	sha256, pbkdf2_hmac	Stores salted hashes of OTPs on the server and slow-hashes the token PIN (100,000 iterations).
secrets (standard library)	token_bytes, token_urlsafe	Cryptographically secure random number generator for keys, challenges, salts and challenge ids.
struct, time, dataclasses	standard library	Byte encoding of counters and timestamps, the clock, and server-side records.
unittest (standard library)	tests/test_otp_auth.py	Automated test framework for the 19 test cases.
Operating system	Windows 10/11, Ubuntu Linux or macOS	Any OS that runs Python 3; development and testing were done on Linux.
IDE / editor	Visual Studio Code or PyCharm	Editing, running and debugging the code.
matplotlib (optional)	report figures only	Drawing the diagrams in this report; not needed to run the project.

3.2 Hardware requirements

Table 2: Hardware requirements

Component	Minimum	Purpose
Processor	Any 1 GHz x86-64 or ARM CPU	HMAC and SHA-256 are lightweight; one verification takes microseconds.
RAM	2 GB	Running Python and the IDE.
Storage	50 MB free	Python interpreter and project files.
Clock	System clock (NTP-synchronised recommended)	Expiry checks and TOTP mode depend on the current time.

No external packages or network connection are required to run the scheme, which keeps the trusted code base small and makes the project easy to reproduce in a college lab.

CHAPTER 4

ALGORITHM / METHODOLOGY AND FLOWCHART

4.1 System architecture

The system has three participants, mirroring the example dynamic password scheme in Section 3.5.2 of the notes: the **token**, which holds the secret key K and the password function and is unlocked with a PIN; the **claimant** (user and client), who relays values between token and server; and the **authentication server** (verifier), which issues challenges and enforces the security rules.

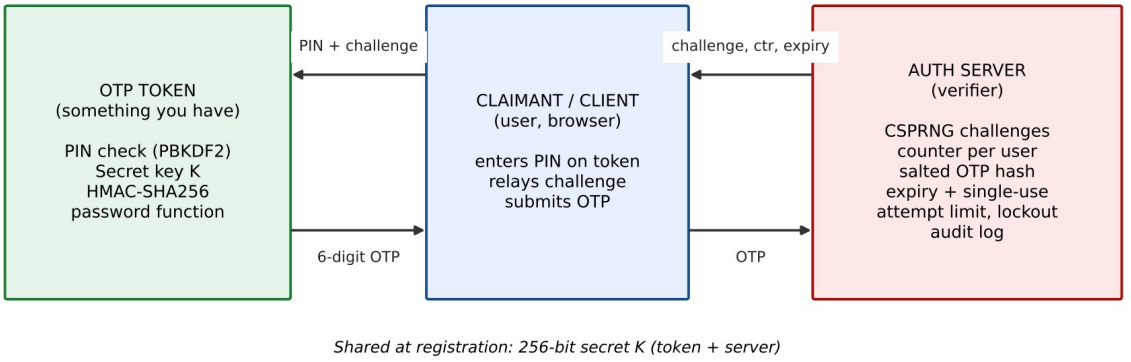


Figure 1: Block diagram of the dynamic OTP authentication system

4.2 Password function (OTP generation)

The password function is an HMAC-based OTP. Given the key K and a message M, it works as follows (RFC 4226 dynamic truncation, using SHA-256 instead of SHA-1):

1. Compute mac = HMAC-SHA256(K, M), a 32-byte value.
2. Take offset = the low 4 bits of the last byte of mac (a value from 0 to 15).
3. Read the 4 bytes mac[offset … offset+3] as a big-endian integer and clear the top bit to get a 31-bit number.
4. OTP = that number mod 10^6, left-padded with zeros to six digits.

In challenge-response mode the message is M = C || n || T, where C is the 16-byte random challenge, n the user's sequence number and T the expiry time (both 8-byte big-endian). Because T is inside the MAC, an attacker cannot lengthen an OTP's lifetime without the OTP changing. In TOTP mode M is the time-step counter floor(t / 30).

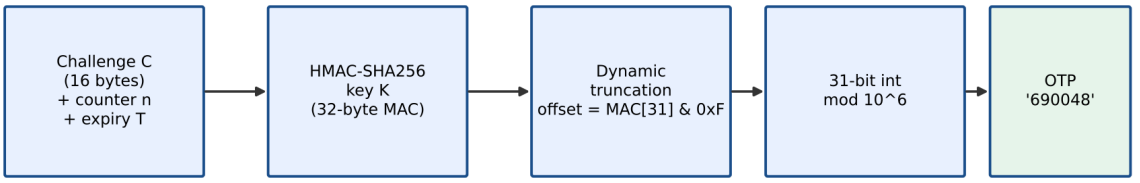


Figure 2: OTP generation using HMAC-SHA256 and dynamic truncation

4.3 Authentication protocol (challenge-response with expiry)

1. **Registration**: the server generates a 256-bit key K with a CSPRNG, stores it in the user record and provisions it into the user's token together with a PIN (stored as a PBKDF2 hash inside the token).
2. **Login request**: the user asks to log in. The server checks that the account is not locked, increments the user's counter n, generates a 128-bit random challenge C, sets T = now + 60 s and computes the expected OTP.
3. **Storage**: the server keeps only {challenge id, C, n, T, salt, SHA-256(salt || OTP), attempts = 0} and sends (challenge id, C, n, T) to the user. The OTP itself is never sent or stored in plain form.
4. **Token unlock**: the user enters the PIN on the token. Without the correct PIN the token refuses to compute anything (two-factor: something you know and something you have).
5. **Response**: the token computes OTP = Trunc(HMAC-SHA256(K, C || n || T)) and displays it; the user submits it.
6. **Verification**: the server applies the checks in Figure 4 in order: account lock, challenge exists for this user, not already used, not expired, well-formed, then a constant-time hash comparison.
7. **Single use**: on success the record is marked used and deleted, so any further submission finds no challenge and is rejected as a replay. On failure the attempt counter grows; at 3 the challenge is revoked, and at 5 failed logins the account is locked for 300 s.

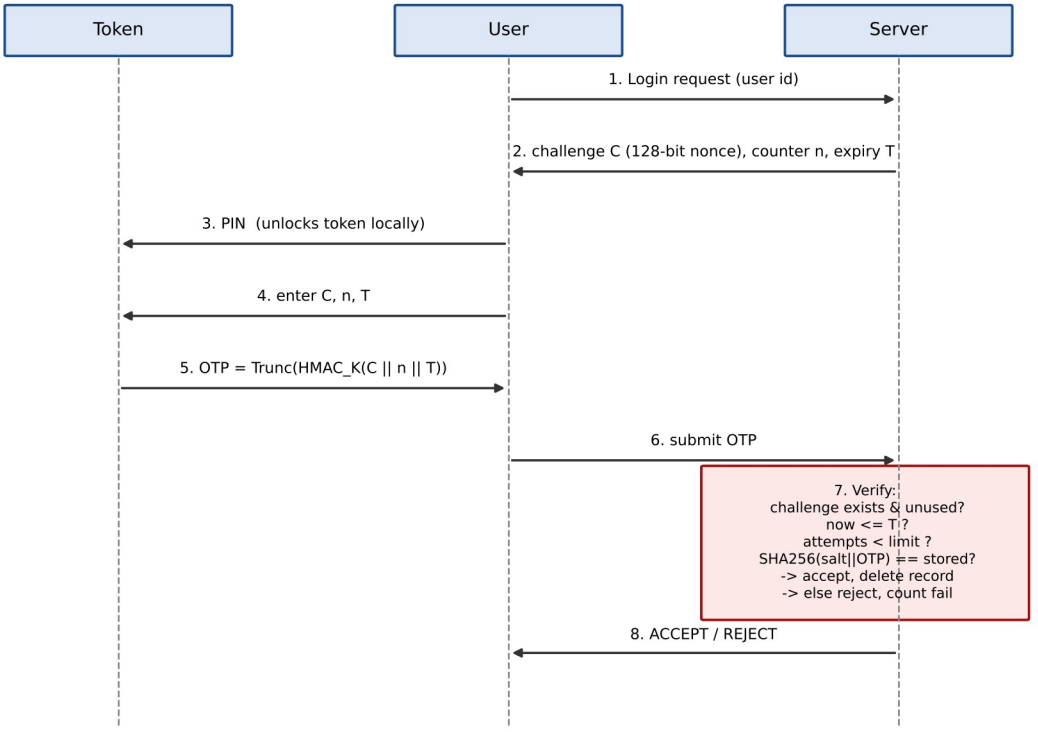


Figure 3: Sequence diagram of one authentication attempt

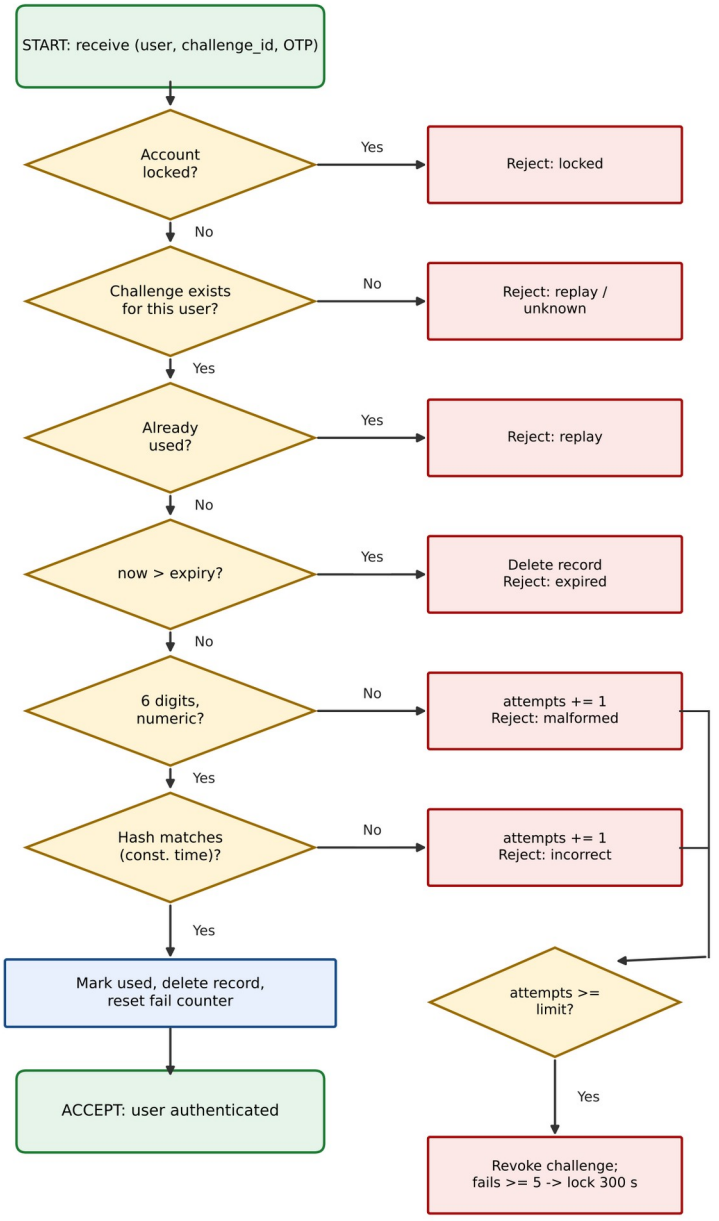


Figure 4: Flowchart of server-side OTP verification

4.4 Time-based mode (TOTP)

For tokens without an input keypad the clock serves as the fresh input. The token shows TOTP = HOTP(K, floor(t / 30)). The server accepts the current step and one step either side (a window of acceptability of about ±30 s, which absorbs clock drift and communication delay as the notes recommend). To make each TOTP single-use, the server records the last accepted time-step for each user and rejects any step that is less than or equal to it. This is the sequence-number freshness rule (accept only if T_new > T_old) applied to time-steps.

CHAPTER 5

IMPLEMENTATION AND SOURCE CODE

5.1 Modules

Table 3: Project modules

File / class	Responsibility
otp_auth.hotp(), hotp_counter(), totp()	Password function: HMAC-SHA256 with dynamic truncation, counter-based HOTP and time-based TOTP.
otp_auth.challenge_message()	Builds the fresh input C || n || T that binds challenge, counter and expiry.
otp_auth.OTPToken	The claimant's token: PIN check with PBKDF2, holds K, computes challenge responses and TOTPs.
otp_auth.AuthServer	The verifier: registration, issue_challenge(), verify(), verify_totp(), attempt limiting, lockout, purge_expired() and an audit log.
otp_auth.Challenge, UserRecord	Server-side data: pending challenge (hash only) and per-user key, counter, failure count, lock time and last TOTP step.
demo.py	Scripted demonstration of 9 scenarios using a simulated clock, plus an --interactive mode on the real clock.
tests/test_otp_auth.py	19 unittest test cases (TC01 to TC19).

5.2 Working procedure

The server object is created with its policy parameters (otp_ttl = 60 s, max_attempts = 3, lockout_threshold = 5, lockout_seconds = 300, totp_window = 1). register() returns an OTPToken that shares the new key. A login is issue_challenge() followed by token.unlock(pin), token.respond(...) and server.verify(...). The clock is passed in as a function, so the tests and the demonstration can move time forward instantly to check expiry without waiting.

5.3 Key source code

Password function (otp_auth.py):

```
def hotp(key: bytes, message: bytes, digits: int = OTP_DIGITS) -> str:
    """HMAC-SHA256 one-time password with RFC 4226 dynamic truncation."""
    mac = hmac.new(key, message, hashlib.sha256).digest()
    offset = mac[-1] & 0x0F                                  # 0..15
    code = struct.unpack(">I", mac[offset:offset + 4])[0] & 0x7FFFFFFF
    return str(code % (10 ** digits)).zfill(digits)

def challenge_message(challenge: bytes, counter: int, expires_at: int) -> bytes:
    # nonce || sequence number || expiry: changing any field changes the OTP
    return challenge + struct.pack(">QQ", counter, expires_at)
```

Issuing a challenge with an expiry time (AuthServer):

```
def issue_challenge(self, user_id: str) -> dict:
    user = self._get_user(user_id)
    now = int(self.clock())
    if now < user.locked_until:
        raise PermissionError(f"account locked for {int(user.locked_until - now)} s")
    user.counter += 1                                    # sequence number
    challenge = secrets.token_bytes(16)                  # 128-bit nonce (CSPRNG)
    expires_at = now + self.otp_ttl                      # window of acceptability
    expected = hotp(user.key, challenge_message(challenge, user.counter, expires_at))
    salt = secrets.token_bytes(16)
    record = Challenge(challenge_id=secrets.token_urlsafe(12), user_id=user_id,
                       challenge_hex=challenge.hex(), counter=user.counter,
                       issued_at=now, expires_at=expires_at, otp_salt=salt,
                       otp_hash=hashlib.sha256(salt + expected.encode()).digest())
```

```
    self.pending[record.challenge_id] = record        # only the HASH is stored
    return {"challenge_id": record.challenge_id, "challenge": record.challenge_hex,
            "counter": record.counter, "expires_at": expires_at}
```

Expiry and single-use verification (AuthServer):

```
def verify(self, user_id: str, challenge_id: str, otp: str) -> VerifyResult:
    user = self._get_user(user_id)
    now = self.clock()
    if now < user.locked_until:
        return self._fail(user, "account locked", count=False)
    record = self.pending.get(challenge_id)
    if record is None or record.user_id != user_id:   # consumed or foreign
        return self._fail(user, "unknown or already-used OTP (replay)")
    if record.used:
        return self._fail(user, "OTP already used (replay)")
    if now > record.expires_at:                        # EXPIRY
        del self.pending[challenge_id]
        return self._fail(user, "OTP expired")
    if not (isinstance(otp, str) and otp.isdigit() and len(otp) == OTP_DIGITS):
        return self._wrong_guess(user, record, "malformed OTP")
    candidate = hashlib.sha256(record.otp_salt + otp.encode()).digest()
    if not hmac.compare_digest(candidate, record.otp_hash):   # constant time
        return self._wrong_guess(user, record, "incorrect OTP")
    record.used = True                                 # SINGLE USE
    del self.pending[challenge_id]
    user.failed_logins = 0
    return VerifyResult(True, "valid, fresh and unused OTP")
```

TOTP with per-step replay protection (AuthServer):

```
def verify_totp(self, user_id: str, otp: str) -> VerifyResult:
    user = self._get_user(user_id)
    current = int(self.clock() // self.totp_step)
    for step in range(current - self.totp_window, current + self.totp_window + 1):
        if hmac.compare_digest(hotp_counter(user.key, step), str(otp)):
            if step <= user.last_totp_step:            # T_new must be > T_old
                return self._fail(user, "TOTP already used (replay)")
            user.last_totp_step = step
            return VerifyResult(True, f"valid TOTP, time-step {step}")
    return self._fail(user, "incorrect or expired TOTP")
```

The complete, commented source code is given in the Appendix and is supplied with this report in the src/ and tests/ folders.

5.4 Sample input and output

```
>>> server = AuthServer(otp_ttl=60)
>>> token  = server.register("alice", pin="4821")
>>> ch = server.issue_challenge("alice")
>>> ch
{'challenge_id': 'mQk3...', 'challenge': '752e2cc28f68b5485832a89999f023ff',
 'counter': 1, 'expires_at': 1760000060}
>>> token.unlock("4821")
True
>>> otp = token.respond(ch["challenge"], ch["counter"], ch["expires_at"])
>>> otp
'690048'
>>> server.verify("alice", ch["challenge_id"], otp)
ACCEPTED (valid, fresh and unused OTP)
>>> server.verify("alice", ch["challenge_id"], otp)       # replay
REJECTED (unknown or already-used OTP (replay))
```

5.5 Instructions to run

1. Install Python 3.9 or later. No other packages are needed.
2. Open a terminal in the project folder otp-project/.

3. Run the demonstration: cd src, then python demo.py
4. Try it yourself with the real clock: python demo.py --interactive
5. Run the test suite from otp-project/: python -m unittest discover -s tests -v

CHAPTER 6

TESTING, RESULTS AND SCREENSHOTS

6.1 Test strategy

Testing was carried out at two levels. The scripted demonstration (demo.py) walks through nine realistic scenarios and prints the server's decision with its reason. The automated suite (19 unittest cases) checks each security property, including the exact expiry boundary, using an injected fake clock so that results are deterministic. The cases cover normal operation, invalid or tampered input, and edge cases as the report format requires. Policy used: OTP lifetime 60 s, 3 attempts per challenge, lockout after 5 failures for 300 s, TOTP step 30 s with a window of ±1 step.

6.2 Test cases

Table 4: Test cases and results

ID	Test case	Input / scenario	Expected output	Actual result
TC01	Normal login	Correct OTP for a fresh challenge	Accepted	**PASS: accepted**
TC02	OTP format and dynamism	50 consecutive logins	6 numeric digits, all different	**PASS: 50 distinct**
TC03	Replay attack	Valid OTP submitted a second time	Rejected (replay)	**PASS: rejected**
TC04	Cross-challenge reuse	OTP of challenge 1 sent for challenge 2	Rejected	**PASS: rejected**
TC05	Expired OTP	Correct OTP submitted after 61 s	Rejected (expired)	**PASS: rejected**
TC06	Edge: exact expiry	Submitted at t = expires_at	Accepted	**PASS: accepted**
TC07	Edge: 1 s after expiry	Submitted at t = expires_at + 1	Rejected	**PASS: rejected**
TC08	Wrong OTP	Correct OTP + 1	Rejected (incorrect)	**PASS: rejected**
TC09	Malformed input	'', 12345, 1234567, abcdef, '12 456'	Rejected (malformed)	**PASS: rejected**
TC10	Tampered expiry	OTP computed with expiry + 3600 s	Rejected	**PASS: rejected**
TC11	Attempt limit	3 wrong guesses, then the correct OTP	Challenge revoked, rejected	**PASS: rejected**
TC12	Account lockout	5 failed logins; retry after 301 s	Locked, then works again	**PASS: as expected**
TC13	Wrong user	Alice's OTP submitted as Bob	Rejected	**PASS: rejected**
TC14	Stolen token	Token used with wrong PIN	Token refuses to compute OTP	**PASS: refused**
TC15	No plaintext storage	Inspect server record	OTP not present	**PASS: hash only**
TC16	TOTP single use	Same TOTP submitted twice	1st accepted, 2nd rejected	**PASS: as expected**
TC17	TOTP expiry	TOTP submitted 90 s later	Rejected	**PASS: rejected**
TC18	Edge: clock drift	Server clock 30 s ahead	Accepted (window ±1)	**PASS: accepted**
TC19	HOTP determinism	Same key and counter; next counter	Same OTP; different OTP	**PASS: as expected**

6.3 Screenshots

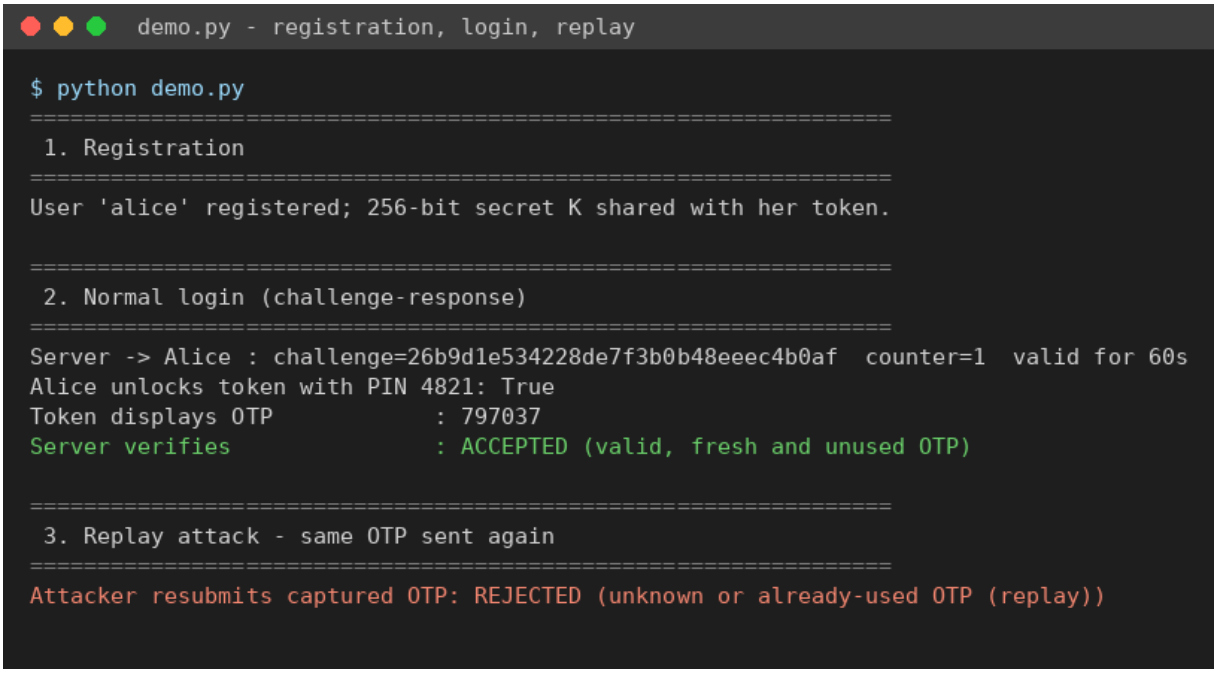


Figure 5: Registration, normal login and replay rejection

Figure 5 shows a complete authentication: the server issues a 128-bit challenge valid for 60 s, the token is unlocked with the PIN and displays a six-digit OTP, and the server accepts it. When the same captured OTP is submitted again the server rejects it, because the challenge record was destroyed on first use.

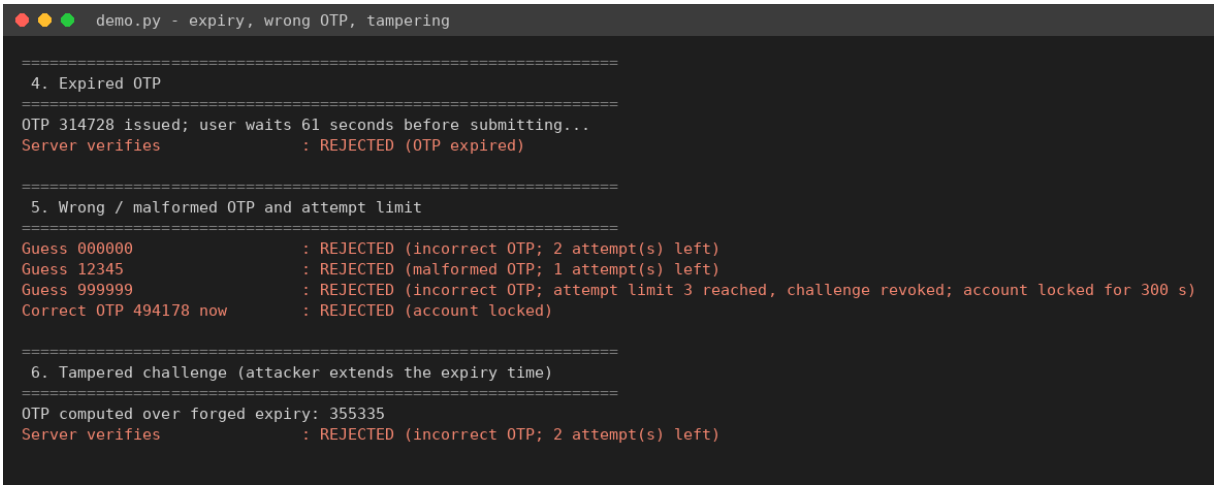


Figure 6: Expired OTP, wrong and malformed OTPs, attempt limit and tampered expiry

Figure 6 shows that a correct OTP submitted 61 seconds later is rejected as expired. Three wrong or malformed guesses revoke the challenge, so even the correct OTP is refused afterwards. An OTP computed over a forged expiry time does not match, which shows the expiry is integrity-protected by the HMAC.

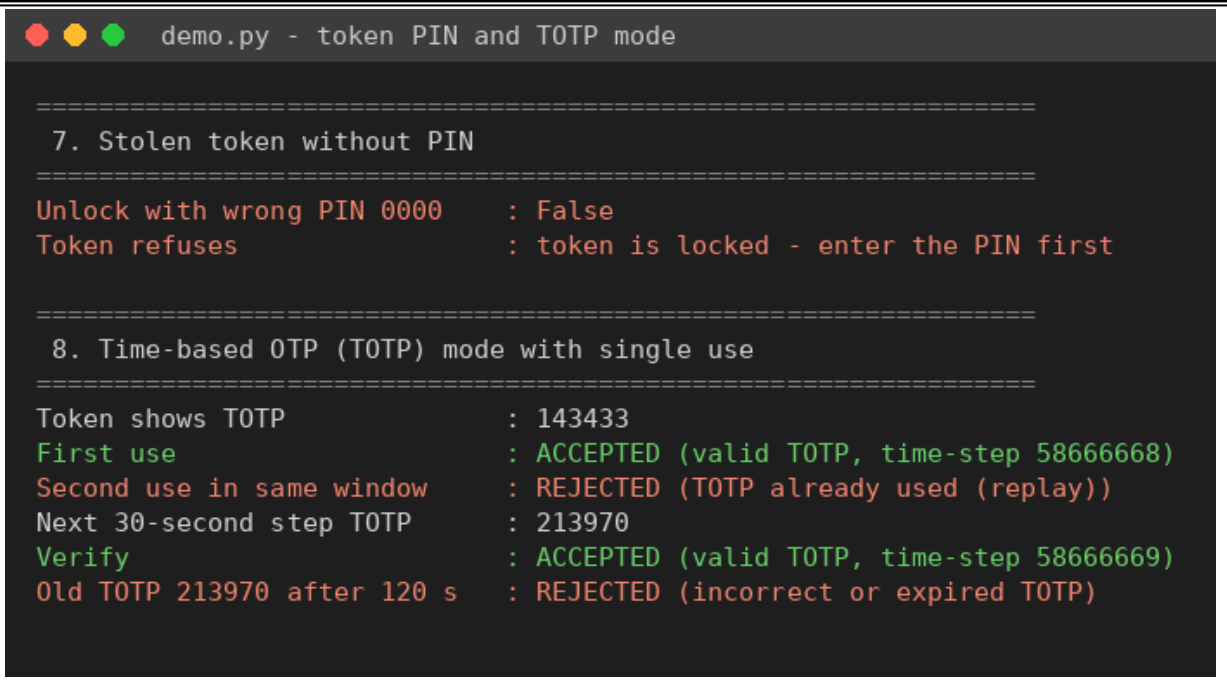


Figure 7: Stolen token without PIN, and TOTP mode single-use behaviour

Figure 7 shows that a token cannot produce an OTP without its PIN. In TOTP mode the first use of a code is accepted and a second use in the same 30-second window is rejected; the next step's code works, and a code that is 120 s old is rejected as expired.

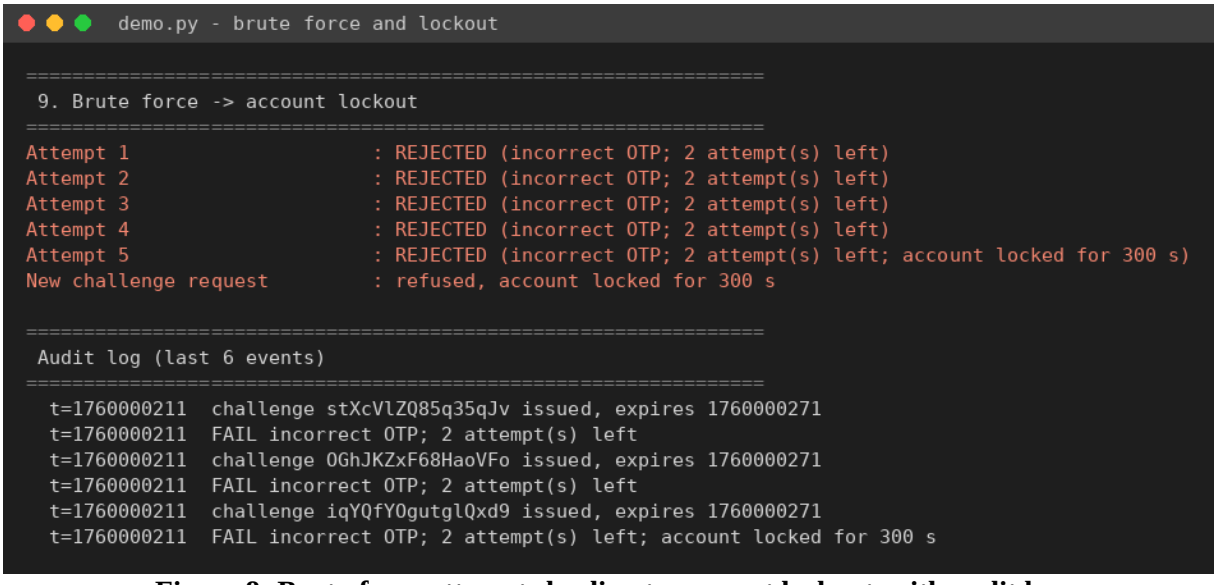


Figure 8: Brute-force attempts leading to account lockout, with audit log

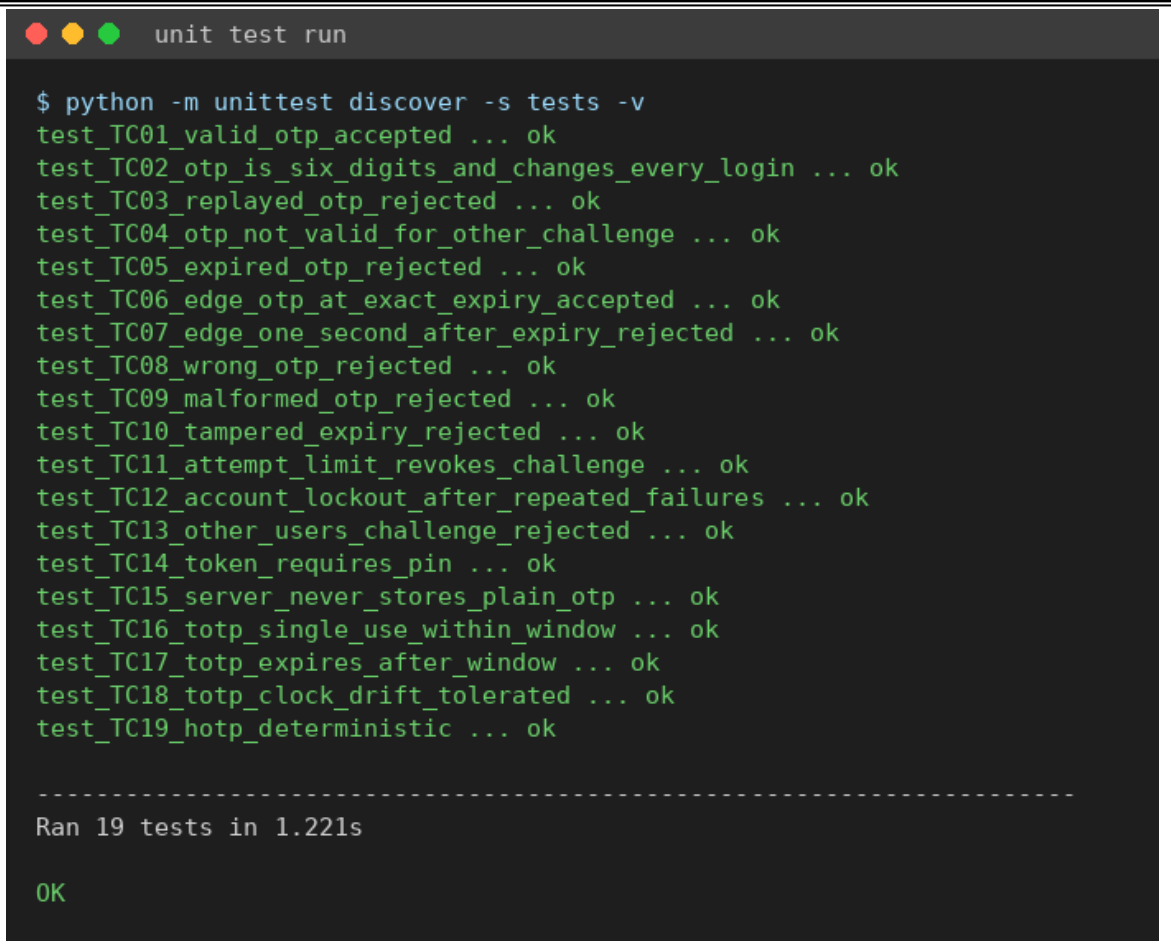


Figure 9: Automated unit test run: 19 tests, all passed

6.4 Interpretation of results

All 19 automated tests passed (Figure 9) and every demonstration scenario produced the expected decision. Each OTP was accepted exactly once and only inside its validity window; the boundary tests (TC06 and TC07) confirm that the window is inclusive of the expiry second and closed one second later. Guessing is bounded: with 3 attempts per challenge and a 300-second lockout after 5 failures, an attacker's chance of guessing a six-digit OTP is at most 3 in 1,000,000 per challenge, and at most 5 guesses per 5 minutes per account.

CHAPTER 7

SECURITY ANALYSIS

7.1 Security services provided

- **Entity authentication**: only a party holding K (inside the token) can compute the correct HMAC response to an unpredictable challenge.
- **Freshness**: three mechanisms from Section 3.2 are combined: a random nonce, a sequence number and an expiry timestamp. An old response cannot match a new challenge, and a response for the current challenge expires within 60 s.
- **Single use / replay protection**: the server deletes the challenge after one success (challenge-response mode) or remembers the last accepted time-step (TOTP mode).
- **Integrity**: challenge, counter and expiry are all inside the HMAC input, so changing any of them, for example to extend the lifetime, gives a different OTP (TC10).
- **Confidentiality of secrets**: K is never transmitted after registration; the server stores only a salted SHA-256 hash of each pending OTP; the PIN never leaves the token and is stored as a PBKDF2 hash.
- **Two-factor authentication**: something the user knows (PIN) and something the user has (token), as in the dynamic-password analysis in the notes.

7.2 Attacks considered

Table 5: Threats and countermeasures

Attack	Countermeasure in this implementation
Replay of a captured OTP	Single-use deletion of the challenge; last-step tracking in TOTP mode.
Delayed use of an intercepted OTP	60-second expiry; T is MAC-protected and cannot be changed.
Brute-force guessing	3 attempts per challenge, account lockout after 5 failures, 10^6 OTP space.
Timing side channel	hmac.compare_digest for all comparisons.
Server database theft	Only salted hashes of short-lived OTPs are stored (keys K remain sensitive, see 7.3).
Predictable challenges	128-bit challenges from the secrets CSPRNG (non-deterministic generator).
Lost or stolen token	PIN required; PIN hashed with PBKDF2 (100,000 iterations).
Using another user's challenge	Challenge is bound to a user id and the user's key.

7.3 Limitations and assumptions

- The 256-bit key K is stored on the server in memory. A real deployment would keep it in a hardware security module or encrypt it with a master key, because whoever steals K can generate valid OTPs.
- Data is held in memory; restarting the program forgets users and pending challenges. A database with atomic "check and delete" is needed in practice so that two parallel requests cannot both use one OTP.
- The channel between client and server is assumed to be protected by TLS. The scheme alone does not stop a real-time man-in-the-middle or phishing site that relays the OTP

within its validity window.

- Account lockout can be abused for denial of service against a known user name; production systems combine it with rate limiting per IP address and CAPTCHA.
- TOTP security depends on reasonably synchronised clocks, as discussed under clock-based mechanisms in the notes.

7.4 Educational simulation versus production security

This project is an educational simulation. The token is a Python object rather than tamper-resistant hardware, the clock can be simulated, and there is no network layer, persistence or secure key storage. The cryptographic choices (HMAC-SHA256, RFC 4226 truncation, CSPRNG randomness, constant-time comparison) are the same as those used in production systems, but before real use the implementation would need secure key provisioning, an HSM, TLS, persistent and concurrency-safe storage, monitoring and an independent security review.

CONCLUSION

The project implemented a dynamic one-time-password authentication scheme based on the challenge-response design in Module 3. All five objectives were met: OTPs are generated with HMAC-SHA256 from a fresh 128-bit challenge and never repeated in testing; each OTP expires after 60 seconds, with the boundary verified to the second; each OTP is accepted at most once, and every replay was rejected; guessing is limited by an attempt counter, account lockout, constant-time comparison and hash-only storage; and a PIN-protected token together with a TOTP mode provides two-factor authentication with drift tolerance and replay protection.

The main observation is that the cryptography alone is not enough. An HMAC-based OTP without server-side expiry and single-use bookkeeping would still be open to replay inside its window. The security comes from combining the password function with the freshness mechanisms (nonce, sequence number and timestamp) and with strict verification rules.

Future enhancements

- A web or mobile front end (for example Flask with a QR-code provisioning page for authenticator apps).
- Persistent storage (SQLite or Redis) with atomic single-use deletion and encrypted keys.
- Out-of-band delivery of OTPs by e-mail or SMS, and push-based approval.
- Mutual authentication, so the server also proves its identity, which helps against phishing.
- Migrating to FIDO2/WebAuthn public-key authentication, which removes shared secrets entirely.

REFERENCES

[1] BIS703 Information & Network Security, Module 3 course notes: Random Number Generation, Providing Freshness, Entity Authentication, Passwords and Dynamic Password Schemes, VTU.

[2] K. M. Martin, Everyday Cryptography: Fundamental Principles and Applications, 2nd ed., Oxford University Press, 2017 (Chapter 8: Entity Authentication).

[3] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache and O. Ranen, "HOTP: An HMAC-Based One-Time Password Algorithm," RFC 4226, IETF, 2005.

[4] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, IETF, 2011.

[5] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, IETF, 1997.

[6] NIST Special Publication 800-63B, Digital Identity Guidelines: Authentication and Lifecycle Management, 2017 (updated).

[7] Python Software Foundation, Python 3 Standard Library documentation: hmac, hashlib, secrets, unittest. https://docs.python.org/3/library/

[8] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed., Pearson, 2023.

CONTRIBUTION OF GROUP MEMBERS

Table 6: Contribution of group members

Member (Name / USN)	Tasks performed
Member 1: STUDENT NAME (USN)	Study of Module 3 dynamic password schemes, design of the protocol and OTP generation (otp_auth.py: hotp, OTPToken), flowchart and sequence diagram, Chapters 1, 2 and 4 of the report.
Member 2: STUDENT NAME (USN)	Server-side verification (expiry, single use, attempt limit, lockout, TOTP mode), demonstration program, unit tests and screenshots, Chapters 3, 5, 6 and 7 of the report.

APPENDIX: COMPLETE SOURCE CODE

A.1 src/otp_auth.py

```
"""
otp_auth.py - Dynamic One-Time-Password (OTP) authentication scheme
with expiry and single-use verification.

Course : BIS703 Information & Network Security (Module 3 - Entity Authentication)

The design follows the "dynamic password scheme" of Module 3 (Section 3.5):
instead of a static password the claimant owns a *password function*
(here HMAC-SHA256 keyed with a secret K, held inside a PIN-protected token)
and a *fresh input* that changes on every attempt.  All three freshness
mechanisms from Section 3.2 are used:

  * Nonce based  : the server issues a random 128-bit challenge per login.
  * Clock based  : every OTP carries an expiry time (window of acceptability).
  * Sequence no. : a per-user counter is mixed into every OTP so that two
                   OTPs are never computed from the same input.

Security rules enforced by the server (the verifier):
  1. Expiry       - an OTP older than `otp_ttl` seconds is rejected.
  2. Single use   - an OTP/challenge is destroyed after the first successful
                    verification; replaying it fails.
  3. Attempt limit- after `max_attempts` wrong guesses the challenge is
                    burnt, and repeated failures lock the account.
  4. No plaintext - the server stores only a salted SHA-256 hash of each OTP
                    and compares in constant time (hmac.compare_digest).

A second mode, time-based OTP (TOTP, RFC 6238), is also provided.  It uses the
clock as the fresh input and remembers the last accepted time-step per user so
that the same TOTP cannot be used twice inside its validity window.

Only the Python standard library is used.
"""

import hashlib
import hmac
import secrets
import struct
import time
from dataclasses import dataclass, field

# ---------------------------------------------------------------------------
# 1. The password function: HMAC-based OTP (RFC 4226 dynamic truncation)
# ---------------------------------------------------------------------------

OTP_DIGITS = 6


def hotp(key: bytes, message: bytes, digits: int = OTP_DIGITS) -> str:
    """Compute an HMAC-SHA256 one-time password over an arbitrary message.

    Steps (RFC 4226, Section 5.3):
      1. mac    = HMAC-SHA256(K, message)               -> 32 bytes
      2. offset = low 4 bits of the last byte           -> 0..15
      3. code   = 31-bit integer from mac[offset:offset+4]
      4. OTP    = code mod 10^digits, left-padded with zeros
    """
    mac = hmac.new(key, message, hashlib.sha256).digest()
    offset = mac[-1] & 0x0F
    code = struct.unpack(">I", mac[offset:offset + 4])[0] & 0x7FFFFFFF
    return str(code % (10 ** digits)).zfill(digits)


def hotp_counter(key: bytes, counter: int, digits: int = OTP_DIGITS) -> str:
    """Classic counter-based HOTP: message is the 8-byte big-endian counter."""
    return hotp(key, struct.pack(">Q", counter), digits)


def totp(key: bytes, at: float, step: int = 30, digits: int = OTP_DIGITS) -> str:
    """Time-based OTP (RFC 6238): the counter is floor(unix_time / step)."""
    return hotp_counter(key, int(at // step), digits)


def challenge_message(challenge: bytes, counter: int, expires_at: int) -> bytes:
    """Fresh input for the challenge-response OTP.

    Binding the nonce, the sequence number and the expiry time together means
    an attacker cannot extend the lifetime or reuse the challenge for another
    login: changing any field produces a different OTP.
    """
    return challenge + struct.pack(">QQ", counter, expires_at)
```

```
# ---------------------------------------------------------------------------
# 2. The claimant's smart token (PIN-protected, holds the secret key K)
# ---------------------------------------------------------------------------

class TokenLockedError(Exception):
    """Raised when the token is used without a correct PIN."""


class OTPToken:
    """Software model of the hardware token in Module 3, Figure 'dynamic
    password scheme based on challenge-response'.

    The PIN never leaves the token; it only unlocks the password function.
    This gives two-factor authentication: something you know (PIN) and
    something you have (the token holding K).
    """

    def __init__(self, user_id: str, key: bytes, pin: str, clock=time.time):
        self.user_id = user_id
        self._key = key
        self._pin_salt = secrets.token_bytes(16)
        self._pin_hash = self._hash_pin(pin)
        self._clock = clock
        self._unlocked = False

    def _hash_pin(self, pin: str) -> bytes:
        return hashlib.pbkdf2_hmac("sha256", pin.encode(), self._pin_salt, 100_000)

    def unlock(self, pin: str) -> bool:
        """Step 2 of the scheme: user authenticates to the token with the PIN."""
        self._unlocked = hmac.compare_digest(self._hash_pin(pin), self._pin_hash)
        return self._unlocked

    def lock(self) -> None:
        self._unlocked = False

    def respond(self, challenge_hex: str, counter: int, expires_at: int) -> str:
        """Step 4: compute the response (the OTP) to a server challenge."""
        if not self._unlocked:
            raise TokenLockedError("token is locked - enter the PIN first")
        msg = challenge_message(bytes.fromhex(challenge_hex), counter, expires_at)
        return hotp(self._key, msg)

    def current_totp(self) -> str:
        """Time-based mode: show the OTP for the current 30-second step."""
        if not self._unlocked:
            raise TokenLockedError("token is locked - enter the PIN first")
        return totp(self._key, self._clock())


# ---------------------------------------------------------------------------
# 3. The verifier: authentication server
# ---------------------------------------------------------------------------

@dataclass
class Challenge:
    """Server-side record of one pending OTP. Only a hash of the OTP is kept."""
    challenge_id: str
    user_id: str
    challenge_hex: str
    counter: int
    issued_at: int
    expires_at: int
    otp_salt: bytes
    otp_hash: bytes
    attempts: int = 0
    used: bool = False


@dataclass
class UserRecord:
    user_id: str
    key: bytes
    counter: int = 0                # sequence number (Section 3.2.2)
    failed_logins: int = 0
    locked_until: float = 0.0
    last_totp_step: int = -1        # replay protection for TOTP mode
    audit: list = field(default_factory=list)


@dataclass
class VerifyResult:
    ok: bool
    reason: str

    def __str__(self) -> str:
        return ("ACCEPTED" if self.ok else "REJECTED") + f" ({self.reason})"
```

```
class AuthServer:
    """Authentication server implementing expiry and single-use verification."""

    def __init__(self, otp_ttl: int = 60, max_attempts: int = 3,
                 lockout_threshold: int = 5, lockout_seconds: int = 300,
                 totp_step: int = 30, totp_window: int = 1, clock=time.time):
        self.otp_ttl = otp_ttl                    # lifetime of an OTP (seconds)
        self.max_attempts = max_attempts          # wrong guesses per challenge
        self.lockout_threshold = lockout_threshold
        self.lockout_seconds = lockout_seconds
        self.totp_step = totp_step
        self.totp_window = totp_window            # +/- steps for clock drift
        self.clock = clock                        # injectable for testing
        self.users: dict[str, UserRecord] = {}
        self.pending: dict[str, Challenge] = {}

    # ---- registration ------------------------------------------------
    def register(self, user_id: str, pin: str) -> OTPToken:
        """Enrol a user: generate a 256-bit secret K, share it with a new token."""
        if user_id in self.users:
            raise ValueError(f"user '{user_id}' already registered")
        key = secrets.token_bytes(32)
        self.users[user_id] = UserRecord(user_id, key)
        return OTPToken(user_id, key, pin, clock=self.clock)

    # ---- challenge-response mode -------------------------------------
    def issue_challenge(self, user_id: str) -> dict:
        """Step 1: generate a fresh random challenge with an expiry time."""
        user = self._get_user(user_id)
        now = int(self.clock())
        if now < user.locked_until:
            raise PermissionError(f"account locked for {int(user.locked_until - now)} s")

        user.counter += 1
        challenge = secrets.token_bytes(16)       # 128-bit nonce from a CSPRNG
        expires_at = now + self.otp_ttl
        expected = hotp(user.key, challenge_message(challenge, user.counter, expires_at))

        salt = secrets.token_bytes(16)
        record = Challenge(
            challenge_id=secrets.token_urlsafe(12),
            user_id=user_id,
            challenge_hex=challenge.hex(),
            counter=user.counter,
            issued_at=now,
            expires_at=expires_at,
            otp_salt=salt,
            otp_hash=hashlib.sha256(salt + expected.encode()).digest(),
        )
        self.pending[record.challenge_id] = record
        self._log(user, f"challenge {record.challenge_id} issued, expires {expires_at}")
        # Only public values are sent to the client; never the OTP itself.
        return {"challenge_id": record.challenge_id, "challenge": record.challenge_hex,
                "counter": record.counter, "expires_at": expires_at}

    def verify(self, user_id: str, challenge_id: str, otp: str) -> VerifyResult:
        """Steps 6-7: check the challenge is still valid, then compare OTPs."""
        user = self._get_user(user_id)
        now = self.clock()

        if now < user.locked_until:
            return self._fail(user, "account locked", count=False)

        record = self.pending.get(challenge_id)
        if record is None or record.user_id != user_id:
            # Unknown, already consumed (single use) or belongs to someone else.
            return self._fail(user, "unknown or already-used OTP (replay)")
        if record.used:
            return self._fail(user, "OTP already used (replay)")
        if now > record.expires_at:
            del self.pending[challenge_id]
            return self._fail(user, "OTP expired")
        if not (isinstance(otp, str) and otp.isdigit() and len(otp) == OTP_DIGITS):
            return self._wrong_guess(user, record, "malformed OTP")

        candidate = hashlib.sha256(record.otp_salt + otp.encode()).digest()
        if not hmac.compare_digest(candidate, record.otp_hash):
            return self._wrong_guess(user, record, "incorrect OTP")

        # Success: mark used and delete so the OTP can never be accepted again.
        record.used = True
        del self.pending[challenge_id]
        user.failed_logins = 0
        self._log(user, f"challenge {challenge_id} verified")
        return VerifyResult(True, "valid, fresh and unused OTP")

    # ---- time-based mode ---------------------------------------------
    def verify_totp(self, user_id: str, otp: str) -> VerifyResult:
        """Verify a TOTP within +/- totp_window steps; each step usable once."""
```

```
        user = self._get_user(user_id)
        now = self.clock()
        if now < user.locked_until:
            return self._fail(user, "account locked", count=False)

        current = int(now // self.totp_step)
        for step in range(current - self.totp_window, current + self.totp_window + 1):
            if hmac.compare_digest(hotp_counter(user.key, step), str(otp)):
                if step <= user.last_totp_step:
                    return self._fail(user, "TOTP already used (replay)")
                user.last_totp_step = step
                user.failed_logins = 0
                self._log(user, f"TOTP step {step} accepted")
                return VerifyResult(True, f"valid TOTP, time-step {step}")
        return self._fail(user, "incorrect or expired TOTP")

    # ---- housekeeping ------------------------------------------------
    def purge_expired(self) -> int:
        """Remove expired challenges (would run periodically in production)."""
        now = self.clock()
        dead = [cid for cid, c in self.pending.items() if now > c.expires_at]
        for cid in dead:
            del self.pending[cid]
        return len(dead)

    def _get_user(self, user_id: str) -> UserRecord:
        if user_id not in self.users:
            raise KeyError(f"unknown user '{user_id}'")
        return self.users[user_id]

    def _wrong_guess(self, user, record, reason) -> VerifyResult:
        record.attempts += 1
        if record.attempts >= self.max_attempts:
            del self.pending[record.challenge_id]   # burn the challenge
            reason += f"; attempt limit {self.max_attempts} reached, challenge revoked"
        else:
            reason += f"; {self.max_attempts - record.attempts} attempt(s) left"
        return self._fail(user, reason)

    def _fail(self, user, reason, count=True) -> VerifyResult:
        if count:
            user.failed_logins += 1
            if user.failed_logins >= self.lockout_threshold:
                user.locked_until = self.clock() + self.lockout_seconds
                reason += f"; account locked for {self.lockout_seconds} s"
        self._log(user, "FAIL " + reason)
        return VerifyResult(False, reason)

    def _log(self, user, event) -> None:
        user.audit.append((round(self.clock(), 2), event))
```

A.2 src/demo.py

```
"""
demo.py - Scripted and interactive demonstration of the dynamic OTP scheme.

Run:
    python demo.py              # scripted walk-through of every scenario
    python demo.py --interactive  # type the OTP yourself (real clock)
"""

import argparse
import time

from otp_auth import AuthServer, TokenLockedError


class SimClock:
    """Simulated clock so that expiry can be shown without waiting."""

    def __init__(self, start: float = 1_760_000_000.0):
        self.now = start

    def __call__(self) -> float:
        return self.now

    def advance(self, seconds: float) -> None:
        self.now += seconds


def banner(text: str) -> None:
    print("\n" + "=" * 64 + f"\n {text}\n" + "=" * 64)


def reset_counters(server, user_id):
    """Clear failure counters between independent demo scenarios."""
    server.users[user_id].failed_logins = 0
```

```
    server.users[user_id].locked_until = 0.0


def scripted() -> None:
    clock = SimClock()
    server = AuthServer(otp_ttl=60, max_attempts=3, lockout_threshold=5, clock=clock)

    banner("1. Registration")
    token = server.register("alice", pin="4821")
    print("User 'alice' registered; 256-bit secret K shared with her token.")

    banner("2. Normal login (challenge-response)")
    ch = server.issue_challenge("alice")
    print(f"Server -> Alice : challenge={ch['challenge']}  counter={ch['counter']}"
          f"  valid for {server.otp_ttl}s")
    print("Alice unlocks token with PIN 4821:", token.unlock("4821"))
    otp = token.respond(ch["challenge"], ch["counter"], ch["expires_at"])
    print(f"Token displays OTP            : {otp}")
    print("Server verifies              :", server.verify("alice", ch["challenge_id"], otp))

    banner("3. Replay attack - same OTP sent again")
    print("Attacker resubmits captured OTP:", server.verify("alice", ch["challenge_id"], otp))

    banner("4. Expired OTP")
    ch = server.issue_challenge("alice")
    otp = token.respond(ch["challenge"], ch["counter"], ch["expires_at"])
    print(f"OTP {otp} issued; user waits 61 seconds before submitting...")
    clock.advance(61)
    print("Server verifies              :", server.verify("alice", ch["challenge_id"], otp))

    banner("5. Wrong / malformed OTP and attempt limit")
    ch = server.issue_challenge("alice")
    good = token.respond(ch["challenge"], ch["counter"], ch["expires_at"])
    for guess in ["000000", "12345", "999999"]:
        print(f"Guess {guess:<7}                :",
              server.verify("alice", ch["challenge_id"], guess))
    print(f"{'Correct OTP ' + good + ' now':<30}:",
          server.verify("alice", ch["challenge_id"], good))

    reset_counters(server, "alice")
    banner("6. Tampered challenge (attacker extends the expiry time)")
    ch = server.issue_challenge("alice")
    forged = token.respond(ch["challenge"], ch["counter"], ch["expires_at"] + 3600)
    print(f"OTP computed over forged expiry: {forged}")
    print("Server verifies              :", server.verify("alice", ch["challenge_id"], forged))

    banner("7. Stolen token without PIN")
    token.lock()
    print("Unlock with wrong PIN 0000    :", token.unlock("0000"))
    try:
        token.respond(ch["challenge"], ch["counter"], ch["expires_at"])
    except TokenLockedError as exc:
        print("Token refuses                :", exc)

    banner("8. Time-based OTP (TOTP) mode with single use")
    token.unlock("4821")
    reset_counters(server, "alice")
    t = token.current_totp()
    print(f"Token shows TOTP              : {t}")
    print("First use                    :", server.verify_totp("alice", t))
    print("Second use in same window    :", server.verify_totp("alice", t))
    clock.advance(30)
    t2 = token.current_totp()
    print(f"Next 30-second step TOTP      : {t2}")
    print("Verify                       :", server.verify_totp("alice", t2))
    clock.advance(120)
    print(f"{'Old TOTP ' + t2 + ' after 120 s':<30}:", server.verify_totp("alice", t2))

    banner("9. Brute force -> account lockout")
    reset_counters(server, "alice")
    for i in range(5):
        ch = server.issue_challenge("alice")
        r = server.verify("alice", ch["challenge_id"], "111111")
        print(f"Attempt {i + 1}                    : {r}")
    try:
        server.issue_challenge("alice")
    except PermissionError as exc:
        print("New challenge request        : refused,", exc)

    banner("Audit log (last 6 events)")
    for ts, event in server.users["alice"].audit[-6:]:
        print(f"  t={ts:.0f}  {event}")


def interactive() -> None:
    server = AuthServer(otp_ttl=60)
    user = input("Register user name: ").strip() or "user"
    pin = input("Choose a token PIN: ").strip() or "1234"
    token = server.register(user, pin)
```

```
    while True:
        cmd = input("\n[l]ogin  [q]uit > ").strip().lower()
        if cmd == "q":
            break
        try:
            ch = server.issue_challenge(user)
        except PermissionError as exc:
            print("Login refused:", exc)
            continue
        print(f"Challenge {ch['challenge']} (expires in {server.otp_ttl}s)")
        if token.unlock(input("Token PIN: ")):
            print("Token OTP:", token.respond(ch["challenge"], ch["counter"], ch["expires_at"]))
        else:
            print("Token: wrong PIN, no OTP shown")
        start = time.time()
        while True:
            entry = input("Enter OTP (blank to stop): ").strip()
            if not entry:
                break
            r = server.verify(user, ch["challenge_id"], entry)
            print(f"[{time.time() - start:.0f}s] {r}")
            if r.ok or "revoked" in r.reason or "expired" in r.reason:
                break
        token.lock()


if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--interactive", action="store_true")
    args = ap.parse_args()
    interactive() if args.interactive else scripted()
```

A.3 tests/test_otp_auth.py

```
"""Unit tests for the dynamic OTP scheme.  Run:  python -m unittest -v"""

import os
import sys
import unittest

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))

from otp_auth import AuthServer, TokenLockedError, hotp_counter  # noqa: E402


class FakeClock:
    def __init__(self, t=1_760_000_000.0):
        self.t = t

    def __call__(self):
        return self.t


class TestOTP(unittest.TestCase):
    def setUp(self):
        self.clock = FakeClock()
        self.server = AuthServer(otp_ttl=60, max_attempts=3, lockout_threshold=5,
                                 lockout_seconds=300, clock=self.clock)
        self.token = self.server.register("alice", pin="4821")
        self.token.unlock("4821")

    def login(self):
        ch = self.server.issue_challenge("alice")
        return ch, self.token.respond(ch["challenge"], ch["counter"], ch["expires_at"])

    # --- normal operation ------------------------------------------------
    def test_TC01_valid_otp_accepted(self):
        ch, otp = self.login()
        self.assertTrue(self.server.verify("alice", ch["challenge_id"], otp).ok)

    def test_TC02_otp_is_six_digits_and_changes_every_login(self):
        otps = {self.login()[1] for _ in range(50)}
        self.assertTrue(all(len(o) == 6 and o.isdigit() for o in otps))
        self.assertGreater(len(otps), 45)       # practically all distinct

    # --- single use / replay ---------------------------------------------
    def test_TC03_replayed_otp_rejected(self):
        ch, otp = self.login()
        self.server.verify("alice", ch["challenge_id"], otp)
        r = self.server.verify("alice", ch["challenge_id"], otp)
        self.assertFalse(r.ok)
        self.assertIn("replay", r.reason)

    def test_TC04_otp_not_valid_for_other_challenge(self):
        ch1, otp1 = self.login()
        ch2, _ = self.login()
```

```
        self.assertFalse(self.server.verify("alice", ch2["challenge_id"], otp1).ok)

    # --- expiry -------------------------------------------------------------
    def test_TC05_expired_otp_rejected(self):
        ch, otp = self.login()
        self.clock.t += 61
        r = self.server.verify("alice", ch["challenge_id"], otp)
        self.assertFalse(r.ok)
        self.assertIn("expired", r.reason)

    def test_TC06_edge_otp_at_exact_expiry_accepted(self):
        ch, otp = self.login()
        self.clock.t = ch["expires_at"]           # boundary: still valid
        self.assertTrue(self.server.verify("alice", ch["challenge_id"], otp).ok)

    def test_TC07_edge_one_second_after_expiry_rejected(self):
        ch, otp = self.login()
        self.clock.t = ch["expires_at"] + 1
        self.assertFalse(self.server.verify("alice", ch["challenge_id"], otp).ok)

    # --- invalid / tampered input -------------------------------------------
    def test_TC08_wrong_otp_rejected(self):
        ch, otp = self.login()
        wrong = str((int(otp) + 1) % 1_000_000).zfill(6)
        self.assertFalse(self.server.verify("alice", ch["challenge_id"], wrong).ok)

    def test_TC09_malformed_otp_rejected(self):
        for bad in ["", "12345", "1234567", "abcdef", "12 456"]:
            ch, _ = self.login()
            r = self.server.verify("alice", ch["challenge_id"], bad)
            self.assertFalse(r.ok)
            self.assertIn("malformed", r.reason)
            self.server.users["alice"].failed_logins = 0

    def test_TC10_tampered_expiry_rejected(self):
        ch = self.server.issue_challenge("alice")
        forged = self.token.respond(ch["challenge"], ch["counter"], ch["expires_at"] + 3600)
        self.assertFalse(self.server.verify("alice", ch["challenge_id"], forged).ok)

    def test_TC11_attempt_limit_revokes_challenge(self):
        ch, otp = self.login()
        for _ in range(3):
            self.server.verify("alice", ch["challenge_id"], "000000")
        r = self.server.verify("alice", ch["challenge_id"], otp)
        self.assertFalse(r.ok)              # even the right OTP is now useless

    def test_TC12_account_lockout_after_repeated_failures(self):
        for _ in range(5):
            ch = self.server.issue_challenge("alice")
            self.server.verify("alice", ch["challenge_id"], "000000")
        with self.assertRaises(PermissionError):
            self.server.issue_challenge("alice")
        self.clock.t += 301                 # lock-out period over
        ch, otp = self.login()
        self.assertTrue(self.server.verify("alice", ch["challenge_id"], otp).ok)

    def test_TC13_other_users_challenge_rejected(self):
        bob = self.server.register("bob", pin="1111")
        ch, otp = self.login()
        self.assertFalse(self.server.verify("bob", ch["challenge_id"], otp).ok)

    def test_TC14_token_requires_pin(self):
        self.token.lock()
        self.assertFalse(self.token.unlock("0000"))
        ch = self.server.issue_challenge("alice")
        with self.assertRaises(TokenLockedError):
            self.token.respond(ch["challenge"], ch["counter"], ch["expires_at"])

    def test_TC15_server_never_stores_plain_otp(self):
        ch, otp = self.login()
        rec = self.server.pending[ch["challenge_id"]]
        self.assertNotIn(otp.encode(), repr(vars(rec)).encode())

    # --- TOTP mode ----------------------------------------------------------
    def test_TC16_totp_single_use_within_window(self):
        t = self.token.current_totp()
        self.assertTrue(self.server.verify_totp("alice", t).ok)
        self.assertFalse(self.server.verify_totp("alice", t).ok)

    def test_TC17_totp_expires_after_window(self):
        t = self.token.current_totp()
        self.clock.t += 90                  # 3 steps later, window is +/-1
        self.assertFalse(self.server.verify_totp("alice", t).ok)

    def test_TC18_totp_clock_drift_tolerated(self):
        t = self.token.current_totp()
        self.clock.t += 30                  # server one step ahead
        self.assertTrue(self.server.verify_totp("alice", t).ok)
```

```
    # --- known-answer test for the HOTP core (RFC 4226 style truncation) ----
    def test_TC19_hotp_deterministic(self):
        key = b"12345678901234567890"
        self.assertEqual(hotp_counter(key, 0), hotp_counter(key, 0))
        self.assertNotEqual(hotp_counter(key, 0), hotp_counter(key, 1))


if __name__ == "__main__":
    unittest.main(verbosity=2)
```